

Lab 2 (1/24/25)

K-Means Clustering

Useful Textbook (K-Means Clustering Section): <https://www.statlearning.com/>

```
In [17]: import numpy as np
import pandas as pd
from numpy import linalg as LA
import matplotlib.pyplot as plt
```

```
In [18]: df = pd.read_csv('https://raw.githubusercontent.com/MMiDS-textbook/MMiDS-textbook.g

# TODO: drop all rows with na values
df = df.dropna()
```

```
In [19]: ## TODO: print the first 5 rows of the dataframe
df.head()
```

```
Out[19]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	39.1	18.7	181.0	3750.0
1	39.5	17.4	186.0	3800.0
2	40.3	18.0	195.0	3250.0
4	36.7	19.3	193.0	3450.0
5	39.3	20.6	190.0	3650.0

```
In [20]: ## TODO Create a variable that lists each column name (in order)
# create column list and assign it to the variable feature_names
#feature_names =
feature_names = df.columns.values.tolist()
feature_names
```

```
Out[20]: ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
```

```
In [21]: ## TODO: turn the pandas df into a numpy array called X
#X = _____
X = df.to_numpy()
X
```

```
Out[21]: array([[ 39.1,  18.7, 181. , 3750. ],
 [ 39.5,  17.4, 186. , 3800. ],
 [ 40.3,  18. , 195. , 3250. ],
 ...,
 [ 50.4,  15.7, 222. , 5750. ],
 [ 45.2,  14.8, 212. , 5200. ],
 [ 49.9,  16.1, 213. , 5400. ]], shape=(342, 4))
```

Lloyd's Algorithm (K-Means Clustering Algorithm)

Code from Chapter 2.3 in the book

Supervised vs Unsupervised Learning

K-means clustering is an unsupervised learning method. Supervised learning uses the labels during training, unsupervised does not have access to the labels during training.

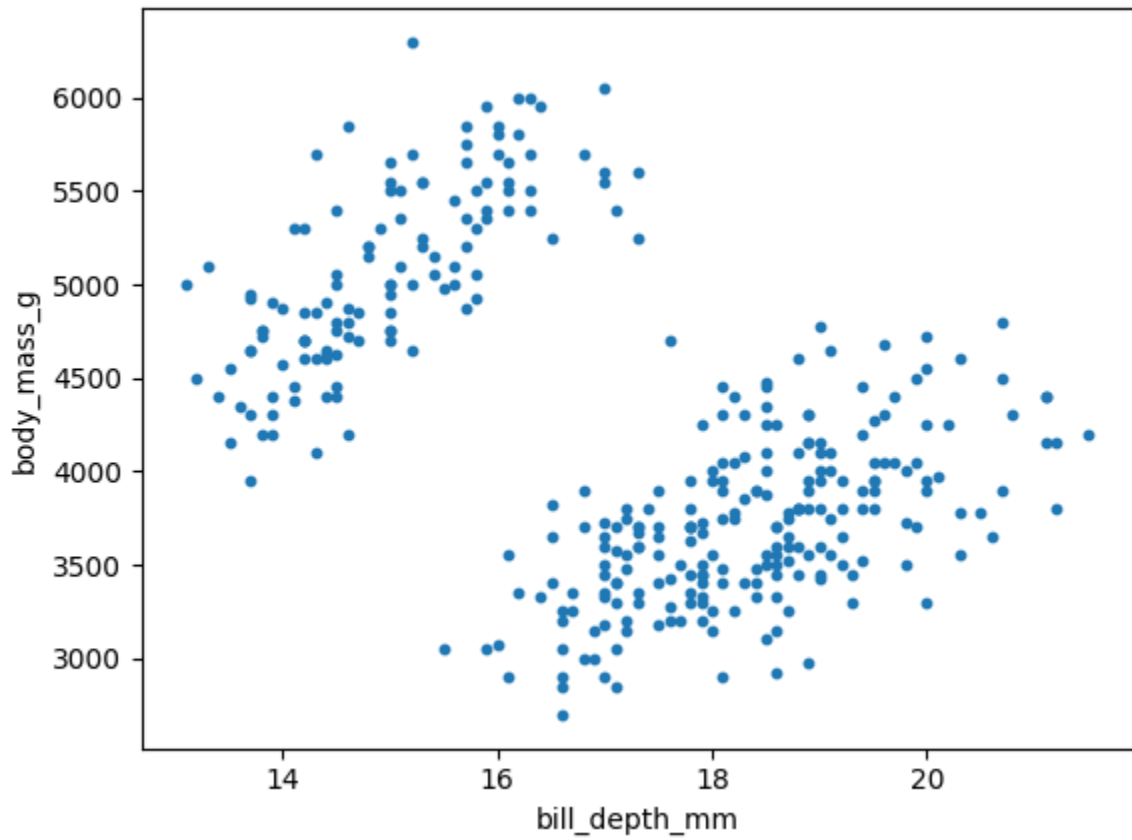
```
In [22]: def opt_reps(X, k, assign):
    (n, d) = X.shape
    reps = np.zeros((k, d))
    for i in range(k):
        in_i = [j for j in range(n) if assign[j] == i]
        reps[i,:] = np.sum(X[in_i,:],axis=0) / len(in_i)
    return reps

def opt_clust(X, k, reps):
    (n, d) = X.shape
    dist = np.zeros(n)
    assign = np.zeros(n, dtype=int)
    for j in range(n):
        dist_to_i = np.array([LA.norm(X[j,:] - reps[i,:]) for i in range(k)])
        assign[j] = np.argmin(dist_to_i)
        dist[j] = dist_to_i[assign[j]]
    G = np.sum(dist ** 2)
    print(G)
    return assign

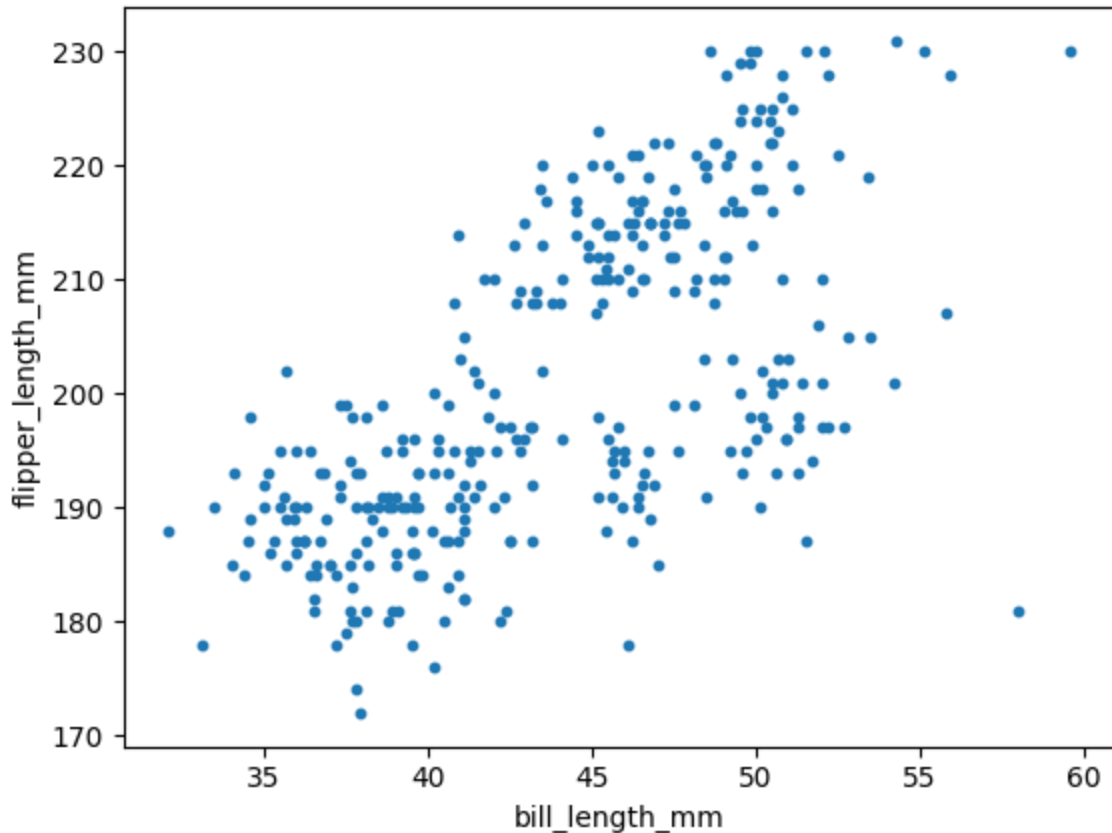
def kmeans(rng, X, k, maxiter=5):
    (n, d) = X.shape
    assign = rng.integers(0,k,n)
    reps = np.zeros((k, d), dtype=int)
    for iter in range(maxiter):
        reps = opt_reps(X, k, assign)
        assign = opt_clust(X, k, reps)
    return assign
```

```
In [23]: # Plot two variables
plt.scatter(X[:,1], X[:,3], s=10)
plt.xlabel('bill_depth_mm')
plt.ylabel('body_mass_g')
```

```
plt.show()
```



```
In [24]: # Plot with two different variables (change axis titles accordingly)
plt.scatter(X[:,0], X[:,2], s=10)
plt.xlabel("bill_length_mm")
plt.ylabel("flipper_length_mm")
plt.show()
```



Knowledge Check: Why do we want to standardize the vectors when we are using k-means clustering? (Hint: Does distance matter?)

```
In [25]: # TODO complete the code below
mean = np.mean(X, axis=0) # Compute mean for each column
std = np.std(X, axis=0) # Compute standard deviation for each column
X[:] = (X - mean) / std # Standardize each column
print(mean, std)

[ 43.92192982  17.15116959 200.91520468 4201.75438596] [ 5.45159602  1.97190392
14.04114057 800.78122924]
```

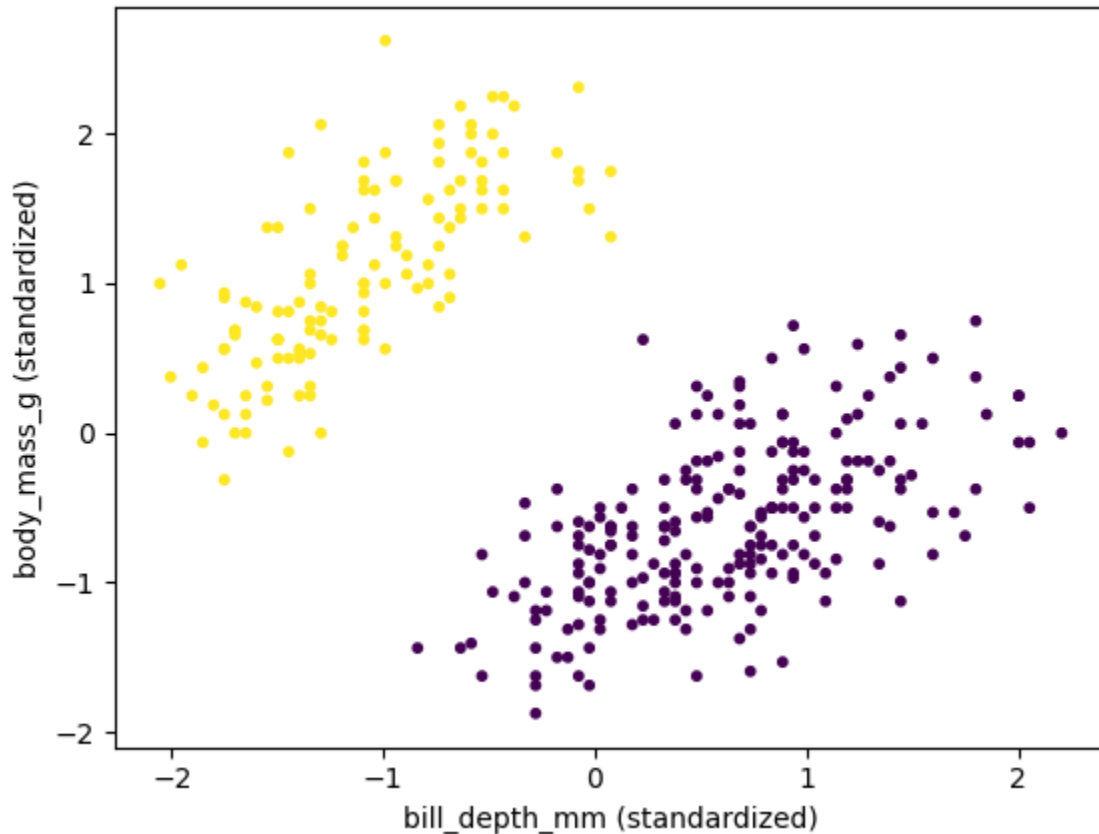
2 Clusters

```
In [26]: #TODO: Check section 1.3.2. for how to assign clusters - do for k=2
seed = 535
rng1 = np.random.default_rng(seed)
assign_2 = kmeans(rng1, X, 2)

1337.3535017777483
578.4819440115109
569.5843202262888
565.821904255223
565.7076453796292
```

```
In [27]: # Color plot with centroid assignment from algorithm
plt.scatter(X[:,1], X[:,3], c=assign_2, s=10)
plt.xlabel('bill_depth_mm (standardized)')
```

```
plt.ylabel('body_mass_g (standardized)')
plt.show()
```

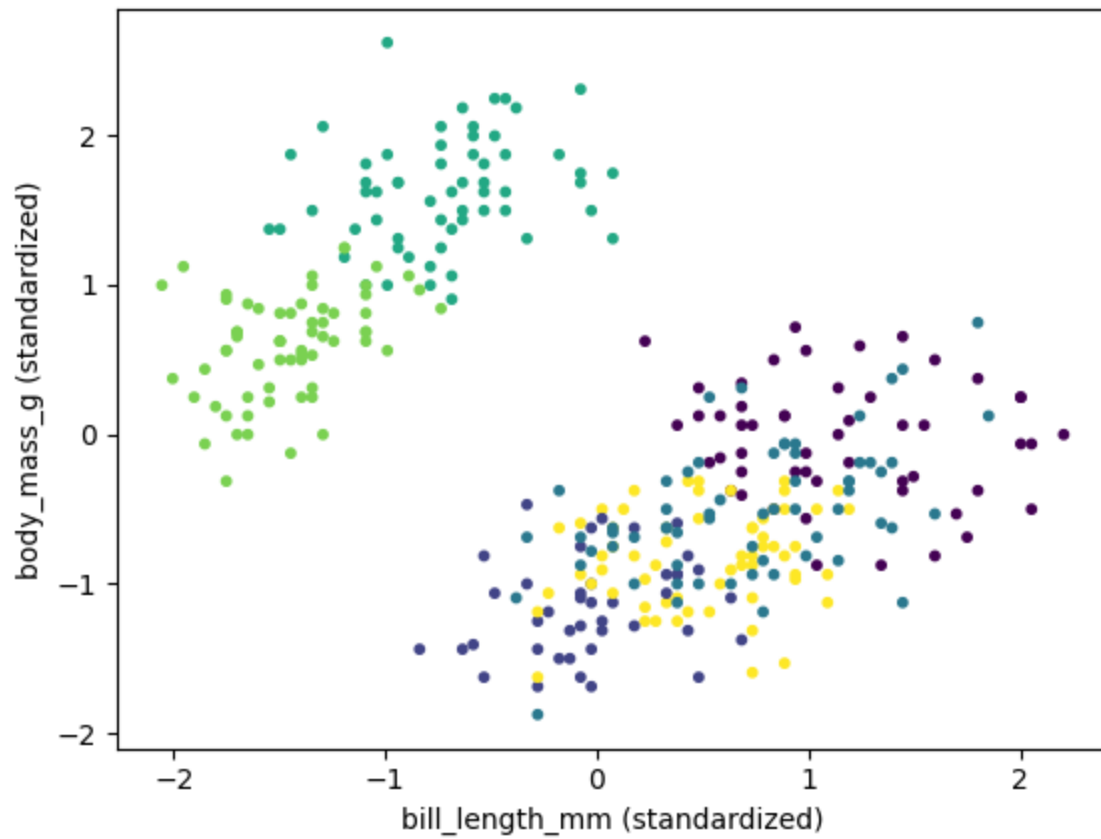


6 Clusters

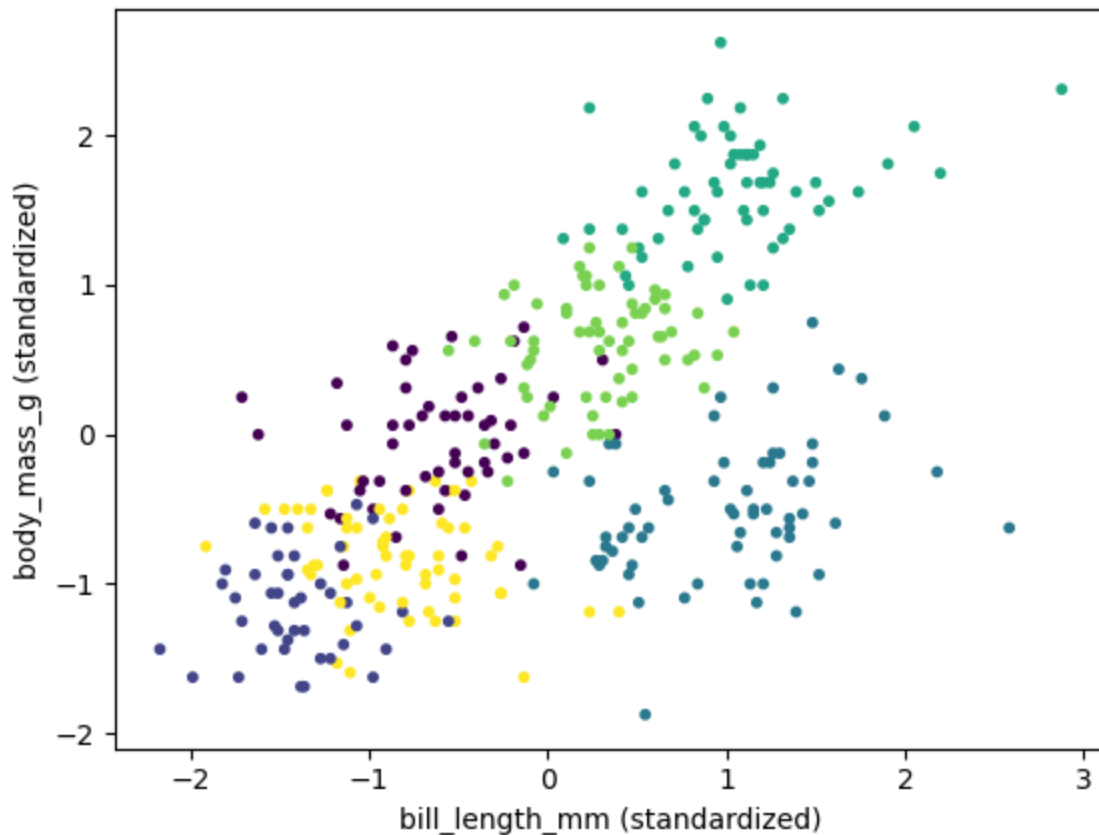
```
In [28]: # TODO: Check section 1.3.2. for how to assign clusters - do for k=6
rng2 = np.random.default_rng(seed)
assign = kmeans(rng2, X, 6)
```

```
965.0631378350881
306.61406877647835
236.17891348679046
220.20857146668973
218.2672534229594
```

```
In [29]: # Notice how the clustering/coloring changes with 6 clusters instead of 2
plt.scatter(X[:,1], X[:,3], c=assign, s=10)
plt.xlabel('bill_length_mm (standardized)')
plt.ylabel('body_mass_g (standardized)')
plt.show()
```



```
In [30]: # Plot a different variable on x
plt.scatter(X[:,0], X[:,3], c=assign, s=10)
plt.xlabel('bill_length_mm (standardized)')
plt.ylabel('body_mass_g (standardized)')
plt.show()
```



View in 3D

In [31]: *# Run this code to produce 3D plots of your features from different angles - you can*

```
def my_scatter(data, colors, labels=feature_names, **kwargs):
    title = kwargs.pop('title', '')
    if title:
        title += ': '

    fig = plt.figure(figsize=(18, 6)) # Adjusted figure size to accommodate 3 plots

    if np.max(colors) != 0:
        normalized_colors = colors/np.max(colors)
    else:
        normalized_colors = colors

    # First subplot: original viewing angle
    ax1 = fig.add_subplot(132, projection='3d', xlabel=labels[0], ylabel=labels[1],
                          zlabel=labels[2], c=normalized_colors, s=20, cmap=cm.get_cmap('magma'))
    ax1.scatter(data[:, 0], data[:, 1], data[:, 2], c=normalized_colors, s=20, cmap=cm.get_cmap('magma'))
    ax1.view_init(elev=30, azim=-45) # Rotate by another 90 degrees (180 degrees total)
    plt.title(f"{title}View from the above-center")

    # Second subplot: rotated view by 90 degrees
    ax2 = fig.add_subplot(133, projection='3d', xlabel=labels[0], ylabel=labels[1],
                          zlabel=labels[2], c=normalized_colors, s=20, cmap=cm.get_cmap('magma'))
    ax2.scatter(data[:, 0], data[:, 1], data[:, 2], c=normalized_colors, s=20, cmap=cm.get_cmap('magma'))
    ax2.view_init(elev=30, azim=ax1.azim+90) # Rotate by 90 degrees
    plt.title(f"{title}View from the above-right")
```

```

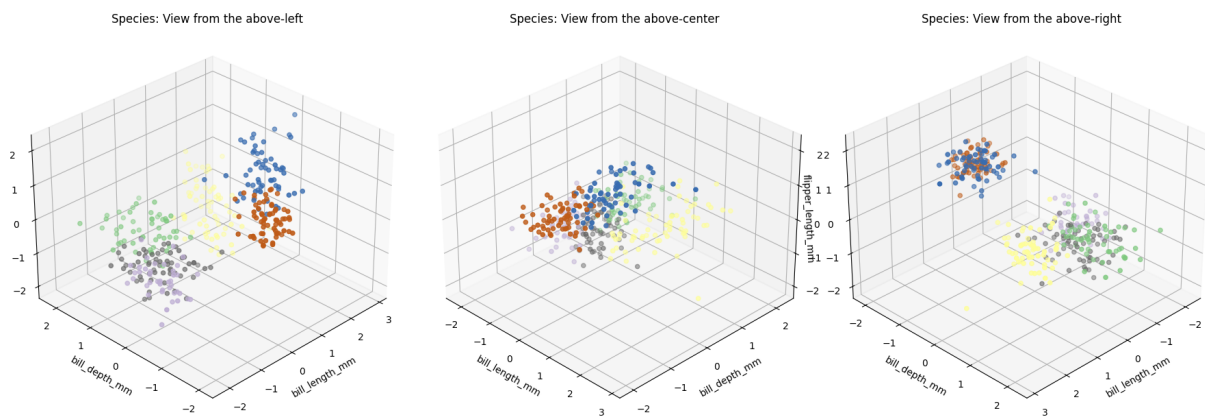
# Third subplot: rotated view by 90 degrees in the opposite direction
ax3 = fig.add_subplot(131, projection='3d', xlabel=labels[0], ylabel=labels[1],
ax3.scatter(data[:, 0], data[:, 1], data[:, 2], c=normalized_colors, s=20, cmap
ax3.view_init(elev=30, azimuth=ax1.azim-90) # Rotate by another 90 degrees
plt.title(f"{title}View from the above-left")

# Display the plot
plt.tight_layout()
plt.show()

return ax1

# Run function
my_scatter(X, assign, title="Species");

```



Elbow Plots

An elbow plot is a graphical tool used to determine the optimal number of clusters in a clustering algorithm, such as K-means. It helps identify the point where adding more clusters no longer significantly improves the clustering performance (because we want clusters that are meaningful and interpretable). This is referred to as the "elbow point."

How Does It Work?

Metric of Interest: The plot displays a metric on the y-axis that measures the clustering performance. For K-means, this metric is usually the sum of squared distances (SSD), also known as "inertia." This measures how tightly the data points are grouped within each cluster.

Number of Clusters: The x-axis represents the number of clusters (k).

Trend: As the number of clusters increases:

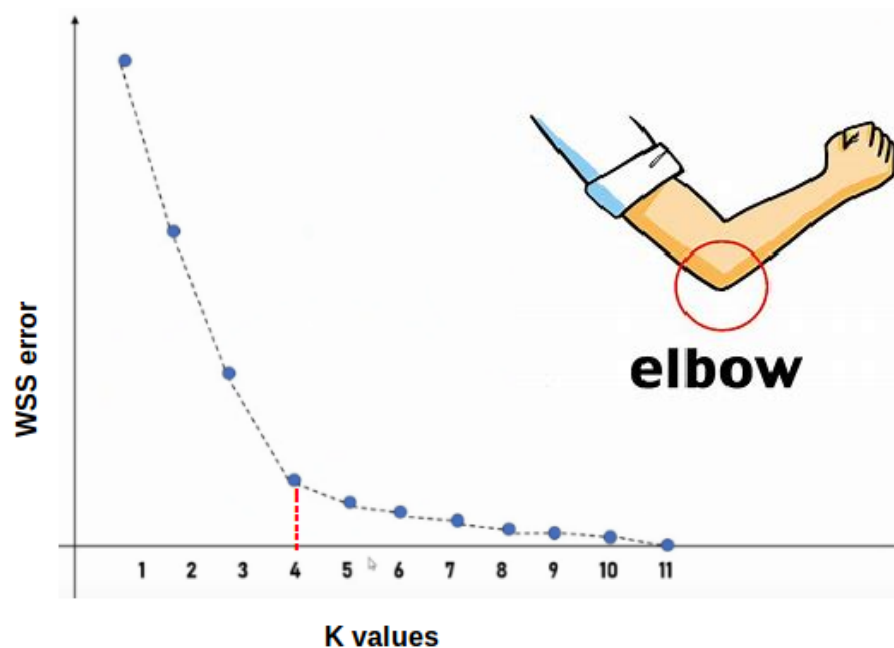
The SSD decreases because each cluster becomes smaller, leading to points being closer to their cluster centroids. Initially, this decrease is rapid because splitting data into more clusters significantly reduces the distances. After a certain point, the decrease slows down, showing diminishing returns.

The Elbow Point

The elbow point is where the SSD curve starts to flatten. This indicates the optimal number of clusters:

Adding more clusters beyond this point does not significantly improve the clustering. It balances between underfitting (too few clusters) and overfitting (too many clusters).

Elbow method



You will be using this method in your Project 1 and comparing it to the silhouette score method for determining the best k

```
In [32]: inertias = []
k_values = range(1, 11)

for k in k_values:
    rng = np.random.default_rng(535)
    labels = kmeans(rng, X, k)

    # Compute Inertia (SSD) manually since kmeans only returns assignment
    centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])
    sq_diff = (X - centroids[labels])**2
    inertia = np.sum(sq_diff)
    inertias.append(inertia)

plt.figure(figsize=(8, 5))
plt.plot(k_values, inertias, marker='o')
```

```
plt.title('Elbow Plot')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (SSD)')
plt.grid(True)
plt.show()
```

```
1368.0000000000002
1368.0000000000002
1368.0000000000002
1368.0000000000002
1368.0000000000002
1337.3535017777483
578.4819440115109
569.5843202262888
565.821904255223
565.7076453796292
1155.7869809776435
591.2533777184822
539.8178471184459
421.41426725519824
382.9497796593495
1022.677652100893
376.2504107176094
345.8042775627591
323.85019293245944
314.11128145798995
1016.7559831569527
381.56062589724036
310.8431310104605
240.54217676186846
234.06712589032173
965.0631378350881
306.61406877647835
236.17891348679046
220.20857146668973
218.2672534229594
851.1013292072985
314.1480311413727
236.46941682234998
196.14548571628097
190.7721991427087
937.8233983885111
285.81548366436834
228.0871216369931
206.03544101838187
191.42304673265602
887.073022355285
289.4757805209356
220.7770920404392
197.68810923176954
189.27821165156556
902.6161063031493
nan
nan
nan
nan
```

```
/tmp/ipykernel_970/3256608733.py:6: RuntimeWarning: invalid value encountered in divide
    reps[i,:] = np.sum(X[in_i,:],axis=0) / len(in_i)
/tmp/ipykernel_970/102440424.py:9: RuntimeWarning: Mean of empty slice
    centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])
/mnt/c/Users/esjmo/nb/.venv/lib/python3.13/site-packages/numpy/_core/_methods.py:134: RuntimeWarning: invalid value encountered in divide
    ret = um.true_divide(
```

